

5CM506 Data driven systems

The Relational Model

Dr. Maqbool Hussain

(m.Hussain@derby.ac.uk)

Aims of the Session

This session aims to help you:

1. Understand the main concepts behind relational databases
2. Explain the theory underpinning relational operations
3. Describe the different levels of a DBMS

Overview

- **The Relational Model**
- Relational Algebra
- Constraints
- Relational DBMS Levels

The Relational Model

Introduced by Codd in 1970 in the paper “*A Relational Model of Data for Large Shared Data Banks*”

- One of the most significant in the history of computer science
- The field of relational database systems originated from this paper

Information Retrieval

P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. CODD
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, shortcomings of these models

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the “connection trap”).

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer presentation are

The Relational Model

The relational data model...

- ...is based on a collection of **relations** and originates from mathematics
 - *Set theory* and *first-order logic* are the main key areas

The Relational Model

The relational data model...

- ...is based on a collection of **relations** and originates from mathematics
 - *Set theory* and *first-order logic* are the main key areas

The underlying principle is that everything stored in a database must be represented by **relations** (two-dimensional table or grid). For example:

The Relational Model

The relational data model...

- ...is based on a collection of **relations** and originates from mathematics
 - *Set theory* and *first-order logic* are the main key areas

The underlying principle is that everything stored in a database must be represented by **relations** (two-dimensional table or grid). For example:

ID	NAME	DEPT_NAME	TOT_CRED
100102030	B. Davis	Computing	180
100102031	M. Nadeem	Computing	180

The Relational Model

The user of the database system may:

- Query the database tables
- Insert new tuples or delete existing tuples
- Modify/Update tuples

The Relational Model

The user of the database system may:

- Query the database tables
- Insert new tuples or delete existing tuples
- Modify/Update tuples

There exist several relational query languages for expressing these operations. The relational query languages define a set of operations on the relations and give tables as output

The Relational Model

The user of the database system may:

- Query the database tables
- Insert new tuples or delete existing tuples
- Modify/Update tuples

There exist several relational query languages for expressing these operations. The relational query languages define a set of operations on the relations and give tables as output

Database products like MS Access, MySQL, PostgreSQL, Oracle Database, MS SQL Server, etc. are inspired by the Relational Model and thus are referred to as Relational Database Management Systems

The Relational Model

- This is not a comprehensive overview of the relational model
- For the sake of brevity, we will cover only some of the most important elements

The Relational Model

- This is not a comprehensive overview of the relational model
- For the sake of brevity, we will cover only some of the most important elements

Complementary to this lecture are Chapter 2 and 6 in the book.

- Chapter 2: Introduction to the Relational Model
- Chapter 6: Formal Relational Query Languages

The Relational Model

- This is not a comprehensive overview of the relational model
- For the sake of brevity, we will cover only some of the most important elements

Complementary to this lecture are Chapter 2 and 6 in the book.

- Chapter 2: Introduction to the Relational Model
- Chapter 6: Formal Relational Query Languages

Some of the topics that are not covered or just in part:

- Predicates
- Set Theory
- First-order logic

Relations

A relation is a **two-dimensional structure** similar to a **table** or **grid**. It is important to note that it is not the same as a SQL table

- A relation has a heading and a body

Relations

A relation is a **two-dimensional** structure similar to a **table** or **grid**. It is important to note that it is not the same as a SQL table

- A relation has a heading and a body

ID (Number)	NAME (Text)	DEPT_NAME (Text)	TOT_CRED (Number)
100102030	B. Davis	Computing	180
100102031	M. Nadeem	Computing	180

Heading

Body

Relations

A relation is a two-dimensional structure similar to a table or grid. It is important to note that it is not the same as a SQL table

- A relation has a heading and a body

ID (Number)	NAME (Text)	DEPT_NAME (Text)	TOT_CRED (Number)
100102030	B. Davis	Computing	180
100102031	M. Nadeem	Computing	180

Heading

Body

- A relation has attributes:
 - The heading of the relation describes the attributes
 - Each **attribute** must have a **name and a type**, both described in the heading
 - Each **name must be unique** in a given heading (some SQL implementations allow duplicated names in the heading)

Relations

- The body of a relation consists of tuples:
 - Each tuple must appear once and only once in a given relation (tuples must be unique (SQL allows duplicate rows))
 - Each tuple must specify a value for each attribute (SQL allows NULL values)
 - Each **value is atomic** or indivisible (may be of any allowable type)
 - All tuple attribute types and names must match the ones in the heading

ID (Number)	NAME (Text)	DEPT_NAME (Text)	TOT_CRED (Number)
100102030	B. Davis	Computing	180
100102031	M. Nadeem	Computing	180

Tuples

Keys

A **superkey** is a set of one or more attributes that, taken collectively, is used to uniquely identify a tuple in a relation

Keys

A **superkey** is a set of one or more attributes that, taken collectively, is used to uniquely identify a tuple in a relation

A **candidate key** is the minimal superkey that is used to uniquely identify a tuple in a relation. There might be multiple candidate keys in a relation

Keys

A **superkey** is a set of one or more attributes that, taken collectively, is used to uniquely identify a tuple in a relation

A **candidate key** is the minimal superkey that is used to uniquely identify a tuple in a relation. There might be multiple candidate keys in a relation

In SQL, one candidate key is given special preference: the **primary key**. This key is chosen by the database designer as the principal means of identifying tuples in a relation

Keys

A **superkey** is a set of one or more attributes that, taken collectively, is used to uniquely identify a tuple in a relation

A **candidate key** is the minimal superkey that is used to uniquely identify a tuple in a relation. There might be multiple candidate keys in a relation

In SQL, one candidate key is given special preference: the **primary key**. This key is chosen by the database designer as the principal means of identifying tuples in a relation

When a relation includes among its attributes the primary key of another relation, we say that this attribute is called a **foreign key** from the first relation referencing the second relation

Relvars

A relation is a special type, like integer, float or string. In a true relational DBMS, we call a variable that holds a relation as **relvar** (**relation-valued variable**)

- A **base relvar** is a variable that is stored in the database until deleted
- A **view** is a relvar whose content is obtained from a query

Relvars

A relation is a special type, like integer, float or string. In a true relational DBMS, we call a variable that holds a relation as **relvar** (**relation-valued variable**)

- A **base relvar** is a variable that is stored in the database until deleted
- A **view** is a relvar whose content is obtained from a query

Base relvars are conceptually equivalent to tables in SQL databases.
Views are conceptually equivalent to a VIEW in SQL databases

In a true relational DBSM, the database is defined entirely by base relvars, and therefore the database is the set of base relvars

Overview

- The Relational Model
- **Relational Algebra**
- Constraints
- Relational DBMS Levels

Relational Algebra

The relational algebra defines a set of operations on relations, analogously to the usual operations in algebra, e.g. addition, subtraction or multiplication

- Every algebraic operation takes relation(s) and produces relation(s)

Relational Algebra

Relational algebra defines a **set of operations on relations**, analogously to the usual operations in algebra, e.g. addition, subtraction, or multiplication

- Every algebraic operation takes relation(s) and produces relation(s)
- Think of it as a normal operation in algebra:

$$3 + 3 = 6$$

$$\text{Integer} \langle \text{operand} \rangle \text{Integer} = \text{Integer}$$

$$\text{Relation} \langle \text{operation} \rangle \text{Relation} = \text{Relation}$$

Relational Algebra

Projection

Outputs specified attributes from all rows of the input relation. Removes duplicate tuples from the output

$$\Pi_{ID, salary}(instructor)$$

ID	name	dept_name	salary
787030	S. Saravi	Computing	90000
787031	L. Stella	Computing	50000
787032	W. Khan	Computing	50000

Relational Algebra

Projection

Outputs specified attributes from all rows of the input relation. Removes duplicate tuples from the output

$$\Pi_{ID, salary}(instructor)$$

Result:

ID	salary
787030	90000
787031	50000
787032	50000

Relational Algebra

Selection/Restriction

Returns rows of the input relation that satisfy the predicate

$$\sigma_{\text{salary} \geq 85000}(\text{instructor})$$

ID	name	dept_name	salary
787030	S. Saravi	Computing	90000
787031	L. Stella	Computing	50000
787032	W. Khan	Computing	50000

Relational Algebra

Selection/Restriction

Returns rows of the input relation that satisfy the predicate

$$\sigma_{\text{salary} \geq 85000}(\text{instructor})$$

Result:

ID	name	dept_name	salary
787030	S. Saravi	Computing	90000

Relational Algebra

Product

Outputs all pairs of rows from the input relations (regardless of whether or not they have the same values on common attributes)

$$r1 \times r2$$

r1

A
a
b

r2

B
1
2
3

Relational Algebra

Product

Outputs all pairs of rows from the input relations (regardless of whether or not they have the same values on common attributes)

$r1 \times r2$

r1		r2		Result:	
A		B		A	B
a		1		a	1
a		2		a	2
a		3		a	3
b		1		b	1
b		2		b	2
b		3		b	3

Relational Algebra

Many more operations:

- **Union**
- **Intersection**
- **Difference**

Relational Algebra

Many more operations:

- **Union**
- **Intersection**
- **Difference**
- ...

Because of the fact that the output of an operation is itself a relation allows us to combine different operations together, e.g.

$$\Pi_{ID,salary}(\sigma_{salary \geq 85000}(instructor))$$

Overview

- The Relational Model
- Relational Algebra
- **Constraints**
- Relational DBMS Levels

Constraints

Among the reasons for using a DBMS there is the fact that we can **maintain the validity (or integrity)** of the data stored in it

This can be achieved through a set of rules, which are called **constraints (or integrity constraints)**. They describe the conditions that must be satisfied in the database by the DBMS

Constraints

Among the reasons for using a DBMS there is the fact that we can **maintain the validity (or integrity)** of the data stored in it

This can be achieved through a set of rules, which are called **constraints (or integrity constraints)**. They describe the conditions that must be satisfied in the database by the DBMS

Examples of constraints are:

- **PRIMARY KEY**
- **FOREIGN KEY**
- **CHECK**

Overview

- The Relational Model
- Relational Algebra
- Constraints
- **Relational DBMS Levels**

Levels of a DBMS

Minimally Relational:

- All data is held in relvars
- The restriction, projection and join operations are supported

Levels of a DBMS

Minimally Relational:

- All data is held in relvars
- The restriction, projection and join operations are supported

Relationally Complete:

- The system has all the properties of a minimally relational DBMS
- A full set of relational algebra operators are supported

Levels of a DBMS

Minimally Relational:

- All data is held in relvars
- The restriction, projection and join operations are supported

Relationally Complete:

- The system has all the properties of a minimally relational DBMS
- A full set of relational algebra operators are supported

Fully Relational:

- The system has all the properties of a relationally complete DBMS
- Integrity constraints are supported and
- User-defined types are supported

Aims of the Session

You should now be able to:

1. Understand the main concepts behind relational databases
2. Explain the theory underpinning relational operations
3. Describe the different levels of a DBMS

References

Silberschatz, A., Korth, H. F., and Sudarshan, S. (2010), *Database System Concepts*, 6th Edition. McGraw-Hill.

- Chapter 2 – Introduction to the Relational Model
- Chapter 6 – Formal Relational Query Languages (Up to Section 6.2)